

Java Design Patterns Interview Questions & MCQs with Answers & Explanations

Concept-Based, Code-Based & Scenario-Based Questions

Q#1. In a chat application, you want users to receive updates about messages in a chat room only when they're subscribed to that room. The system should also manage the communication between different chat rooms. Which combination of patterns is suitable? *(Scenario-based, Multi-select)*

- A) Observer Pattern
- B) Adapter Pattern
- C) Chain of Responsibility Pattern
- D) Mediator Pattern

Answer: A) Observer Pattern and D) Mediator Pattern

Explanation:

- **(A) Observer Pattern: Correct.** The Observer pattern suits the need for notifying subscribers of updates in a chat room.
- **(B) Adapter Pattern: Incorrect.** Given scenario doesn't satisfy the features of Adapter pattern.
- **(C) Chain of Responsibility Pattern: Incorrect.** Given scenario doesn't satisfy the features of the Chain of Responsibility pattern.
- **(D) Mediator Pattern: Correct.** Mediator pattern facilitates coordinated communication among different chat rooms.

Q#2. There are four statements about the participants involved in the context of Adapter Pattern.

1. **Target:** The domain-specific class that the client uses.
2. **Adapter:** A class that implements the Target interface and adapts the Adaptee to it.
3. **Adaptee:** An existing class with an incompatible interface that needs adapting.
4. **Client:** The class that interacts with the Target class.

Which of the following option is correct? *(Concept-based, Single-select)*

- A) Statements 1 & 2 are correct
- B) Statements 1 & 3 are correct

- C) Statements 2 & 3 are correct
- D) All statements are correct

Answer: C

Explanation:

- **Statement 2 (Correct):** Adapter is a class that implements the Target interface and adapts the Adaptee to it.
- **Statement 3 (Correct):** Adaptee is an existing class with an incompatible interface that needs adapting.
- **Statement 1 (Incorrect):** Target is the domain-specific interface that the client uses. It's not a class, but interface.
- **Statement 4 (Incorrect):** Client is the class that interacts with the Target interface, but not the Target class.

Q#3. Consider the following code snippet. What design pattern does this code demonstrate?

(Code-based, Single-select)

```
public class CPU {
    public void start() {
        System.out.println("CPU started.");
    }
}

public class Memory {
    public void load() {
        System.out.println("Memory loaded.");
    }
}

public class HardDrive {
    public void read() {
        System.out.println("Hard drive reading.");
    }
}

public class Computer {
    private CPU cpu;
    private Memory memory;
    private HardDrive hardDrive;

    public Computer() {
        this.cpu = new CPU();
    }
}
```

```

        this.memory = new Memory();
        this.hardDrive = new HardDrive();
    }

    public void startComputer() {
        cpu.start();
        memory.load();
        hardDrive.read();
    }
}

```

- A) Prototype
- B) Builder
- C) Mediator
- D) Facade

Answer: D) Façade

Explanation:

- **D) Facade: Correct.** The Facade pattern simplifies a complex system by providing a unified interface. Here, ComputerFacade starts the system with simplified calls to CPU, Memory, and HardDrive.
- **A) Prototype: Incorrect.** Prototype clones existing objects, unrelated to simplifying complex systems as in Facade.
- **B) Builder: Incorrect.** Builder constructs complex objects step-by-step, which isn't represented here.
- **C) Mediator: Incorrect.** Mediator centralizes interactions between objects, unlike the unifying interface shown here.

Q#4. An e-commerce platform wants to implement a product recommendation system that:

1. **Provides Real-Time Recommendations:** As users browse, the system recommends similar items based on categories like "frequently bought together," "similar items," and "recently viewed."
2. **Allows Custom Recommendation Algorithms:** Developers can plug in various recommendation algorithms without altering the main recommendation flow, such as collaborative filtering, content-based filtering, and user-based filtering.
3. **Logs Recommendation Events:** The system logs user interactions with recommended items (e.g., clicks, add-to-cart actions) to analyze engagement data, and it should be extensible to add more logging features without modifying existing code.

Which design pattern(s) would be most appropriate to implement this recommendation system?

(Scenario-based, Multi-select)

- A) Strategy Pattern
- B) Decorator Pattern
- C) Chain of Responsibility Pattern
- D) Observer Pattern

Answer: A) Strategy Pattern and B) Decorator Pattern

Explanation:

- **A) Strategy Pattern: Correct.** The Strategy pattern is perfect for situations where multiple algorithms can be used interchangeably. Here, different recommendation algorithms (like collaborative, content-based, and user-based filtering) can be designed as separate strategies, allowing developers to change them without modifying the main recommendation engine. This keeps the recommendation flow flexible and adaptable.
- **B) Decorator Pattern: Correct.** The Decorator pattern allows for dynamically adding behaviours to objects, such as logging additional data or adding new features without altering existing code. In this case, as new logging requirements are introduced, the Decorator pattern provides a structured way to extend logging features for recommendation events without modifying the core recommendation classes, enhancing maintainability.
- **C) Chain of Responsibility Pattern: Incorrect.** Although the Chain of Responsibility pattern allows handling requests across multiple handlers, it isn't ideal for scenarios that require dynamic selection of algorithms or the addition of logging functionalities. This pattern wouldn't meet the system's goals as effectively as the Strategy and Decorator patterns.
- **D) Observer Pattern: Incorrect.** The Observer pattern is used to notify multiple objects about changes in another object's state, but it doesn't suit the need to select recommendation algorithms or enhance logging capabilities for this recommendation system.

This scenario illustrates how the Strategy pattern offers flexible algorithm selection while the Decorator pattern allows seamless addition of features, meeting both the recommendation and logging needs of the system.

Q#5. Analyse the following code snippet.

```
public class Singleton {  
    private static Singleton instance;
```

```

private Singleton() {}

public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class) {
            if (instance == null) {
                instance = new Singleton();
            }
        }
    }
    return instance;
}
}

```

Which of the following approach is being used to ensure a thread-safe Singleton implementation?

(Code-based, Single-select)

- A) Eager Initialization
- B) Synchronized Method
- C) Double-Checked Locking
- D) Holder Class (Bill Pugh Method)

Answer: C) Double-Checked Locking

Explanation:

- **A) Eager Initialization:** *Incorrect.* Eager initialization creates the Singleton instance as soon as the class is loaded, without any lazy loading.
- **B) Synchronized Method:** *Incorrect.* A synchronized method locks the entire method, whereas in this code only a block within `getInstance` is synchronized.
- **C) Double-Checked Locking: Correct.** The code checks twice if `instance` is null, synchronizing only the first time a thread tries to initialize it.
- **D) Holder Class (Bill Pugh Method):** *Incorrect.* This approach uses a static inner class for lazy initialization, not double-checked locking.

Q#6. In which of the following situations is the Builder pattern particularly useful?

(Concept-based, Multi-select)

- A) When the object construction process involves multiple steps
- B) When there are several optional parameters in an object
- C) When the object type varies based on the input provided
- D) When a single constructor with multiple parameters would be confusing

Answer: A) When the object construction process involves multiple steps; B) When there are several optional parameters in an object; D) When a single constructor with multiple parameters would be confusing

Explanation:

- **A) Multiple construction steps: Correct.** The Builder pattern is ideal for scenarios with multiple construction steps.
- **B) Optional parameters: Correct.** The Builder pattern allows for optional parameters without numerous constructors.
- **C) Varying object types: Incorrect.** This is better suited for Factory Method, not Builder.
- **D) Complex constructor parameters: Correct.** Builder pattern helps avoid large, confusing constructors.

Q#7. Analyze the following Singleton implementation. Which aspect of it ensures thread safety in a multithreaded environment?

(Code-based, Single-select)

```
public class Singleton {  
    private static volatile Singleton instance;  
  
    private Singleton() {}  
  
    public static Singleton getInstance () {  
        if (instance == null) {  
            synchronized (Singleton.class) {  
                if (instance == null) {  
                    instance = new Singleton ();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

- A) The synchronized block
- B) The volatile keyword
- C) The private constructor
- D) Both synchronized block and volatile keyword

Answer: D) Both synchronized block and volatile keyword

Explanation:

- **A) synchronized block:** *Partially correct.* While `synchronized` blocks control access to initialization, they work with the `volatile` keyword to ensure full thread safety.
- **B) volatile keyword:** *Partially correct.* `volatile` ensures visibility of changes across threads but cannot by itself prevent multiple threads from entering the block initially.
- **C) Private constructor:** *Incorrect.* The private constructor only prevents external instantiation, not multithreaded issues.
- **D) Both synchronized block and volatile keyword: Correct.** Together, `synchronized` and `volatile` ensure thread-safe Singleton instantiation in a double-checked locking approach.

Q#8. Which of the following additions would you make to the following `HouseBuilder` class to ensure it follows the Builder pattern correctly?
(Code-based, Single-select)

```
public class HouseBuilder {
    private int rooms;
    private boolean hasGarage;
    private boolean hasSwimmingPool;

    public HouseBuilder setRooms(int rooms) {
        this.rooms = rooms;
        return this;
    }

    public HouseBuilder setGarage(boolean hasGarage) {
        this.hasGarage = hasGarage;
        return this;
    }

    public HouseBuilder setSwimmingPool(boolean hasSwimmingPool) {
        this.hasSwimmingPool = hasSwimmingPool;
        return this;
    }
}
```

- A) Add a `House` class to be built and a `build()` method that returns a `House` object
- B) Add a `HouseBuilderInterface` to define the build methods
- C) Create a `Director` class to initialize `HouseBuilder`
- D) Replace method chaining with a constructor

Answer: A) Add a House class to be built and a build() method that returns a House object

Explanation:

- **A) House class with build(): Correct.** A build() method is essential to the Builder pattern, returning the final object.
- **B) HouseBuilderInterface: Incorrect.** An interface isn't strictly required for this implementation.
- **C) Director class: Incorrect.** While useful, a Director class is optional.
- **D) Constructor without chaining: Incorrect.** Removing chaining would go against the Builder's design.

Q#9. You are building a report generation tool with various formats like PDF, CSV, and XML. Each format has specific formatting rules, but all follow a general generation process. Which design pattern should you use?

(Scenario-based, Single-select)

- A) Template Method Pattern
- B) Observer Pattern
- C) Builder Pattern
- D) Singleton Pattern

Answer: A) Template Method Pattern

Explanation:

- **(A) Template Method Pattern: Correct.** Template Method allows general process control with format-specific steps.
- **(B) Observer Pattern: Incorrect.** Observer monitors changes, not suitable for process definition.
- **(C) Builder Pattern: Incorrect.** Builder constructs complex objects but doesn't define process templates.
- **(D) Singleton Pattern: Incorrect.** Singleton controls instances, irrelevant to templated steps.

Q#10. Which of the following are correctly matched with their respective Design Pattern categories?

(Concept-based, Multi-select)

- A) Structural pattern: Flyweight, Adapter, Proxy
- B) Creational pattern: Factory Method, Builder, Memento
- C) Behavioral pattern: Observer, Chain of Responsibility, Command
- D) Behavioral pattern: Template Method, Iterator, Decorator

Answer: A) and C)

Explanation:

- **(A)** Flyweight, Adapter, Proxy: **Correct.** These are all Structural Patterns.
- **(C)** Observer, Chain of Responsibility, Command: **Correct.** These are [Behavioral Patterns](#).
- **(B)** Factory Method, Builder, Memento: **Incorrect.** Memento is a Behavioral Pattern, while the others are [Creational Patterns](#).
- **(D)** Template Method, Iterator, Decorator: **Incorrect.** Decorator is a [Structural Pattern](#), not Behavioral.

Q#11. Imagine you are developing a logging utility for an application that should have only one logging instance throughout the application lifecycle. The application runs in a multithreaded environment. Which approach of the Singleton Pattern should be the best fit to avoid synchronization overhead and ensure thread safety? (*Scenario-based, Single-select*)

- A) Double-Checked Locking
- B) Synchronized Method
- C) Holder Class (Bill Pugh Method)
- D) Enum Singleton

Answer: C) Holder Class (Bill Pugh Method)

Explanation:

- **A) Double-Checked Locking:** *Incorrect.* While it does reduce synchronization overhead, double-checked locking is more complex and can still have some edge cases if not implemented carefully.
- **B) Synchronized Method:** *Incorrect.* Synchronizing the entire method can lead to performance bottlenecks in a high-concurrency environment.
- **C) Holder Class (Bill Pugh Method):** **Correct.** The Holder Class method uses a static inner class, providing a clean and efficient thread-safe Singleton without explicit synchronization.
- **D) Enum Singleton:** *Incorrect.* Although effective, Enum Singleton is often overkill for most cases and is less flexible compared to the Holder Class method.

Q#12. What technique should be used in a Singleton class to prevent multiple instances from being created when the object is serialized and then deserialized? (*Concept-based, Single-select*)

- A) Implement a private constructor
- B) Mark the instance variable as volatile

- C) Implement the readResolve method
- D) Use double-checked locking

Answer: C) Implement the readResolve method

Explanation:

- **A) Private constructor:** *Incorrect.* While it restricts instantiation from outside the class, it doesn't prevent multiple instances created through deserialization.
- **B) Marking instance as volatile:** *Incorrect.* This helps with visibility in a multithreaded environment but does not address serialization issues.
- **C) readResolve method:** **Correct.** Implementing [readResolve](#) ensures that upon deserialization, the existing instance is returned, maintaining the Singleton guarantee.
- **D) Double-checked locking:** *Incorrect.* Double-checked locking is a thread-safety mechanism for lazy initialization but does not prevent Singleton violations due to deserialization.

Q#13. Which of the following conclusion is correct on the below Singleton implementation?

(Code-based, Single-select)

```
public class Singleton {
    private static Singleton instance;

    private Singleton() { }

    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

- A) This implementation is thread-safe and efficient due to synchronization.
- B) This implementation is not thread-safe because of potential race conditions.
- C) This implementation is thread-safe but may cause performance issues in high-concurrency environments.
- D) This implementation is unsafe for multithreading and should be changed to a static block.

Answer: C) This implementation is thread-safe but may cause performance issues in high-concurrency environments.

Explanation:

- **A) Thread-safe and efficient due to synchronization:** *Incorrect.* While the synchronized method is thread-safe, it can become a bottleneck in high-concurrency scenarios because only one thread can access it at a time.
- **B) Not thread-safe due to race conditions:** *Incorrect.* Synchronization in `getInstance` prevents race conditions, ensuring thread safety.
- **C) Thread-safe but may cause performance issues: Correct.** Synchronizing the entire method limits access, potentially slowing performance if many threads call `getInstance` simultaneously.
- **D) Unsafe and requires a static block:** *Incorrect.* A static block is unnecessary; synchronization here is sufficient for thread safety.

Q#14. Consider the following code snippet. Which statement is correct about this implementation?

```
public class Singleton {  
    private static final Singleton instance = new Singleton();  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        return instance;  
    }  
}
```

- A) This is a correct implementation of an eager-initialized, thread-safe Singleton.
- B) This implementation is thread-unsafe because it does not synchronize `getInstance`.
- C) This implementation is lazy-loaded, so it won't work in multithreaded environments.
- D) This Singleton implementation is incorrect because it allows multiple instances through reflection.

Answer: A) This is a correct implementation of an eager-initialized, thread-safe Singleton.

Explanation:

- **A) This is a correct implementation of an eager-initialized, thread-safe Singleton: Correct.** Eager initialization creates the instance when the class loads, ensuring thread safety without additional synchronization.
- **B) Thread-unsafe due to lack of synchronization: Incorrect.** Since eager initialization creates the instance immediately, synchronization in getInstance is unnecessary.
- **C) Lazy-loaded and unsuitable for multithreading: Incorrect.** Eager initialization means the instance is created at load time, not lazily.
- **D) Incorrect due to reflection vulnerability: Incorrect.** While reflection can indeed break Singleton, this does not invalidate the pattern itself.

Q#15. Consider a Singleton class that uses eager initialization. Which of the following approaches can help prevent the Singleton instance from being created multiple times through reflection?

1. Implementing a check inside the constructor to throw an exception if the instance is already created.
2. Using synchronized blocks to control instantiation.
3. Marking the instance final.
4. Using Enum Singleton.

Options:

- A) Statements 1 & 2
- B) Statements 1 & 4
- C) Statements 3 & 4
- D) All statements are correct

Answer: B) Statements 1 & 4

Explanation:

- **Statement 1: Correct.** Adding a check inside the constructor to throw an exception on additional instance creation can help control Singleton violations through reflection.
- **Statement 2: Incorrect.** Synchronization alone cannot prevent reflection-based instantiation.
- **Statement 3: Incorrect.** Marking the instance as final does not prevent reflection attacks from creating new instances.
- **Statement 4: Correct.** Enum Singleton is inherently protected against reflection violations, making it an ideal choice.

Q#16: Which of the following best describes the main purpose of the Builder design pattern?

- A) To define a family of algorithms and make them interchangeable
- B) To allow for complex object construction by providing a step-by-step approach
- C) To restrict instantiation of a class to a single instance
- D) To enable subclasses to alter the type of objects created by a superclass

Answer: B) To allow for complex object construction by providing a step-by-step approach

Explanation:

- **A) Define a family of algorithms:** *Incorrect.* This describes the Strategy pattern.
- **B) Step-by-step object construction:** **Correct.** The Builder pattern is designed to simplify the construction of complex objects through step-by-step creation, allowing for more flexible configurations.
- **C) Single instance restriction:** *Incorrect.* This describes the Singleton pattern.
- **D) Altering object types in subclasses:** *Incorrect.* This is associated with the Factory Method pattern.

Q#17. Which of the following statements are true about the Builder pattern?
(Select all that apply)

- A) The Builder pattern helps separate object construction from its representation.
- B) It is useful for constructing objects that require many parameters.
- C) The Builder pattern is commonly used to ensure a single instance of a class.
- D) The Builder pattern enables method chaining.

Answer: **A)** The Builder pattern helps separate object construction from its representation; **B)** It is useful for constructing objects that require many parameters; **D)** The Builder pattern enables method chaining

Explanation:

- **A) Separation of construction and representation:** **Correct.** Builder allows this separation, making object creation more flexible.
- **B) Many parameters:** **Correct.** Builder pattern is ideal for complex objects with numerous parameters.
- **C) Single instance guarantee:** *Incorrect.* This is a characteristic of the Singleton pattern.

- **D) Method chaining: Correct.** Builder pattern often supports chaining, allowing flexible object construction.

Q#18. A developer needs to build a `Pizza` object with various options, like crust type, toppings, size, and extra cheese. The developer wants to avoid a large number of constructors for each configuration. Which design pattern would be ideal for this scenario?

- A) Factory
- B) Singleton
- C) Builder
- D) Prototype

Answer: C) Builder

Explanation:

- **A) Factory: Incorrect.** The Factory pattern is mainly for creating different types of objects rather than configuring complex objects with multiple parameters.
- **B) Singleton: Incorrect.** Singleton ensures a single instance, not multiple configurations.
- **C) Builder: Correct.** The Builder pattern is ideal here as it provides a flexible way to construct complex objects like `Pizza` with different attributes.
- **D) Prototype: Incorrect.** Prototype is used for cloning objects rather than building complex configurations.

Q#19. You are developing a software system that needs to generate multiple types of reports (e.g., PDF, Excel, HTML). The specific report type required is only known at runtime, and each report type has a distinct creation process. Which of the following reasons justify using the Factory pattern in this scenario? (Scenario-based, Multi-select)

- A) It allows for dynamic creation of report types at runtime.
- B) It minimizes dependencies by hiding report creation details from the client.
- C) It ensures all reports follow a single instantiation process, even if they differ significantly.
- D) It allows switching report types without modifying client code.

Answer: A) It allows for dynamic creation of report types at runtime; B) It minimizes dependencies by hiding report creation details from the client; D) It allows switching report types without modifying client code.

Explanation:

The Factory pattern is appropriate here because it:

- **A) Correct. Allows dynamic creation** of different report types based on runtime information.
- **B) Correct. Encapsulates creation details** so that the client doesn't need to know the specifics of each report's instantiation.
- **D) Correct. Allows switching report types without altering client code**, fostering flexibility and reducing maintenance.
- **C) Option C is incorrect**, as the Factory pattern allows each report type to have its own instantiation process.

Q#20. In the Factory pattern, which of the following best describes the purpose of a Factory class?

- A) It is responsible for creating objects of subclasses based on conditions without exposing the exact subclass instantiation details.
- B) It stores all instances of subclasses created in the application for reference.
- C) It manages the lifecycle of objects, including cleanup and memory deallocation.
- D) It provides a global point of access to all instances created in the application.

Answer: A) It is responsible for creating objects of subclasses based on conditions without exposing the exact subclass instantiation details.

Explanation:

In the Factory pattern, the Factory class abstracts the object creation process, allowing for flexible instantiation of different subclasses without exposing specific subclass details. Options **B**, **C**, and **D** refer to other patterns or concepts, such as object pooling or singleton access, which are unrelated to the Factory pattern.

Q#21. Which of the following best describes the role of an Abstract Factory in the Abstract Factory Pattern?

- A. It defines a factory interface for creating a single product
- B. It allows for the creation of families of related or dependent objects without specifying their concrete classes
- C. It implements a single factory for all product types in the application
- D. It uses prototyping to generate objects based on a predefined model

Answer: B. It allows for the creation of families of related or dependent objects without specifying their concrete classes

Explanation:

- **A. Incorrect.** An Abstract Factory produces families of products, not just a single product.
- **B. Correct.** This is the defining characteristic of the Abstract Factory Pattern.
- **C. Incorrect.** The Abstract Factory typically defines multiple interfaces for different families of products.
- **D. Incorrect.** The Prototype Pattern, not Abstract Factory, is used for cloning objects.

Q#22. You are implementing a travel booking system where each booking type (flights, hotels) has varying methods. You need a flexible way to switch between booking providers for each type. Which pattern should you use?

- A) Abstract Factory Pattern
- B) Decorator Pattern
- C) Singleton Pattern
- D) Chain of Responsibility Pattern

Answer: A) Abstract Factory Pattern

Explanation:

- **(A) Abstract Factory Pattern: Correct.** Abstract Factory allows creating related objects for each provider.
- **(B) Decorator Pattern:** Incorrect. Decorator adds behaviour without managing object creation.
- **(C) Singleton Pattern:** Incorrect. Singleton limits instances but isn't ideal for managing providers.
- **(D) Chain of Responsibility Pattern:** Incorrect. Chain handles sequential requests, not flexible creation.

Q#23. What is the primary purpose of the Prototype design pattern?

- A) To create a single instance of a class
- B) To create a copy of an existing object with minimal overhead
- C) To define an interface for creating families of related objects
- D) To separate object construction from its representation

Answer: B) To create a copy of an existing object with minimal overhead

Explanation:

- **A) Single instance:** *Incorrect.* This describes the Singleton pattern.

- **B) Copy of an existing object: Correct.** Prototype is designed for creating clones of existing objects to avoid the cost of creating them from scratch.
- **C) Families of related objects: Incorrect.** This is the goal of the Abstract Factory pattern.
- **D) Separate construction from representation: Incorrect.** This describes the Builder pattern.

Q#24. In a scenario where multiple instances of similar objects are required, why might the Prototype pattern be preferred over directly using constructors?
(Select all that apply)

- A) It reduces the overhead of creating objects with complex initialization
- B) It makes adding subclasses easier
- C) It ensures the creation of only one instance of the object
- D) It supports cloning objects with various configurations

Answer: A) It reduces the overhead of creating objects with complex initialization; D) It supports cloning objects with various configurations

Explanation:

- **A) Reduces overhead: Correct.** Cloning an existing object can be more efficient than complex instantiation.
- **B) Adding subclasses: Incorrect.** Prototype doesn't directly aid subclassing.
- **C) Single instance: Incorrect.** This is a characteristic of Singleton.
- **D) Cloning with configurations: Correct.** Prototype allows the creation of objects with different configurations.

Q#25. What is the primary purpose of the Adapter Pattern?

- A) To create an interface for incompatible classes to work together
- B) To add additional functionality to an object dynamically
- C) To restrict object creation to a single instance
- D) To provide a simplified interface to a complex system

Answer: A

Explanation:

- **A (Correct):** The **Adapter Pattern** allows incompatible interfaces to work together by creating an intermediary adapter.
- **B (Incorrect):** This describes the Decorator Pattern.
- **C (Incorrect):** This describes the Singleton Pattern.
- **D (Incorrect):** This describes the Facade Pattern.

Q#26. Which of the following roles does NOT directly participate in the Adapter Pattern?

- A) Adapter
- B) Target
- C) Client
- D) Factory

Answer: D

Explanation:

- **D (Correct):** The **Factory** is not part of the Adapter Pattern structure.
- **A, B, and C (Incorrect):** The **Adapter**, **Target**, and **Client** are essential roles in the Adapter Pattern.

Q#27. You have an old logging system that outputs logs in XML format, but your application now uses JSON for all logs. How can the Adapter Pattern help here?

- A) It can convert XML logs to JSON format without changing the old logging system
- B) It replaces the old logging system with a JSON-based one
- C) It combines XML and JSON logs into a single file
- D) It updates the old logging system to output JSON directly

Answer: A

Explanation:

- **A (Correct):** The **Adapter Pattern** allows the old logging system to work with JSON by translating XML output to JSON.
- **B (Incorrect):** Replacing the system does not involve adapting the interface.
- **C (Incorrect):** This does not address compatibility.
- **D (Incorrect):** Updating the system isn't necessary when using an adapter.

Q#28. What is the primary purpose of the Composite Pattern?

- A) To allow an object to change its behaviour at runtime
- B) To compose objects into tree structures to represent part-whole hierarchies
- C) To provide a simplified interface for complex subsystems
- D) To define a family of algorithms and encapsulate each one

Answer: B

Explanation:

- **B (Correct):** The Composite Pattern is designed to compose objects into tree structures, representing part-whole hierarchies effectively.
- **A (Incorrect):** Changing behaviour at runtime describes the Strategy Pattern.
- **C (Incorrect):** Simplifying complex subsystems relates to the Facade Pattern.
- **D (Incorrect):** Encapsulating algorithms is the essence of the Strategy Pattern.

Q#29. In the Composite Pattern, which of the following is true about the leaf and composite components?

- A) Only the leaf can hold state
- B) Both leaf and composite can implement the same interface
- C) Composite cannot contain leaf nodes
- D) Leaf nodes are responsible for managing child nodes

Answer: B

Explanation:

- **B (Correct):** Both leaf and composite components typically implement the same interface, allowing clients to interact with them uniformly.
- **A (Incorrect):** Both can hold state; it's not limited to just the leaf.
- **C (Incorrect):** Composites are designed specifically to contain leaf nodes.
- **D (Incorrect):** Only composite nodes are responsible for managing child nodes.

Q#30. A graphical application allows users to create shapes, groups of shapes, and nested groups of shapes. Which design pattern would best suit this scenario?

- A) Composite Pattern
- B) Singleton Pattern
- C) Adapter Pattern
- D) Observer Pattern

Answer: A

Explanation:

- **A (Correct):** The Composite Pattern is ideal for representing a tree structure of objects, such as shapes and groups of shapes.
- **B (Incorrect):** The Singleton Pattern restricts instantiation, which is not relevant here.

- **C (Incorrect):** The Adapter Pattern is used for converting interfaces, not for hierarchical structures.
- **D (Incorrect):** The Observer Pattern is focused on event handling, not composition.

Q#31. What is the primary purpose of the Proxy Pattern in software design?

- A) To provide a simplified interface to complex subsystems.
- B) To control access to an object by creating a surrogate.
- C) To manage the lifecycle of objects in a system.
- D) To optimize the performance of an application.

Answer: B

Explanation:

- **B (Correct):** The Proxy Pattern creates a surrogate or placeholder for another object to control access to it.
- **A (Incorrect):** This describes the Facade Pattern, not the Proxy Pattern.
- **C (Incorrect):** Managing the lifecycle of objects is not the primary purpose of the Proxy Pattern.
- **D (Incorrect):** While performance can be optimized, it is not the main focus of this pattern.

Q#32. In the following code, which class represents the Proxy in the Proxy pattern?

```
public class ImageHelper implements Image {
    private ReallImage reallImage;

    public void display() {
        if (reallImage == null) {
            reallImage = new ReallImage();
        }
        reallImage.display();
    }
}
```

- A) ReallImage
- B) ImageHelper
- C) Image
- D) display

Answer: B

Explanation:

- **B is correct** as ImageHelper controls access to RealImage, deferring its instantiation.
- **A is incorrect** since RealImage is the actual object, not the proxy.
- **C is incorrect** because it is the interface type, not a specific implementation.
- **D is incorrect** as it is a method, not a class.

Q#33. What is the primary purpose of the Flyweight Pattern?

- A) To minimize memory usage by sharing common data.
- B) To increase the complexity of code.
- C) To provide a way to create complex objects.
- D) To ensure object immutability.

Answer: A

Explanation:

- **A (Correct):** The Flyweight Pattern aims to minimize memory usage by sharing common data among multiple objects.
- **B (Incorrect):** It simplifies memory management, not increases complexity.
- **C (Incorrect):** It focuses on sharing common data rather than creating complex objects.
- **D (Incorrect):** Immutability is not a defined feature of the Flyweight Pattern.

Q#34. Which of the following components are typically part of the Flyweight Pattern? (Select all that apply)

- A) Flyweight Interface
- B) Concrete Flyweight
- C) Client
- D) Flyweight Factory

Answer: A, B, C, D

Explanation:

- **A (Correct):** The Flyweight Interface defines the operations for Flyweight objects.
- **B (Correct):** Concrete Flyweights implement the Flyweight interface and manage shared state.
- **C (Correct):** The Client uses Flyweights to perform operations.
- **D (Correct):** The Flyweight Factory creates and manages Flyweight objects.

Q#35. What is the primary purpose of the Facade Pattern?

- A) To reduce the complexity of a system by providing a simplified interface.
- B) To allow a class to change its behaviour when its internal state changes.
- C) To create an interface for creating families of related or dependent objects.
- D) To define a one-to-many dependency between objects.

Answer: A

Explanation:

- **A (Correct):** The primary purpose of the Facade Pattern is to provide a simplified interface to a complex subsystem.
- **B (Incorrect):** This describes the State Pattern.
- **C (Incorrect):** This describes the Abstract Factory Pattern.
- **D (Incorrect):** This describes the Observer Pattern.

Q#36. Which of the following is a benefit of using the Facade Pattern?

- A) Increased performance due to reduced object creation.
- B) Improved maintainability of the codebase by minimizing dependencies.
- C) Easier implementation of the Singleton Pattern.
- D) Simplification of data validation mechanisms.

Answer: B

Explanation:

- **B (Correct):** The Facade Pattern improves maintainability by minimizing the dependencies between the client code and the complex subsystem.
- **A (Incorrect):** Performance is not a primary benefit.
- **C (Incorrect):** The Singleton Pattern is unrelated to the Facade Pattern.
- **D (Incorrect):** Data validation is not a concern of the Facade Pattern.

Q#37. Which of the following best describes the purpose of the Bridge Pattern?

- A) To combine two unrelated interfaces into a single unit
- B) To separate an object's abstraction from its implementation
- C) To provide a simplified interface to a complex subsystem
- D) To allow creation of families of related objects

Answer: B

Explanation:

- **A (Incorrect):** Combining unrelated interfaces is a concept closer to the Adapter Pattern.
- **B (Correct):** The **Bridge Pattern** is designed to **decouple abstraction and implementation**, allowing them to vary independently.
- **C (Incorrect):** Simplifying an interface relates to the Facade Pattern.
- **D (Incorrect):** Creating related object families aligns with the Abstract Factory Pattern.

Q#38. In the Bridge Pattern, which component typically contains a reference to the implementor interface?

- A) ConcreteImplementor
- B) RefinedAbstraction
- C) Abstraction
- D) Bridge

Answer: C

Explanation:

- **A (Incorrect):** ConcreteImplementor is a specific implementation of the implementor interface.
- **B (Incorrect):** RefinedAbstraction extends Abstraction but does not typically contain the reference.
- **C (Correct):** The Abstraction class has a reference to the implementor interface, which allows abstraction to delegate operations to the implementor.
- **D (Incorrect):** "Bridge" is not a class but rather describes the connection between abstraction and implementor.

Q#39. Which of the following statements correctly describes the primary purpose of the Decorator Pattern?

- A) To add new responsibilities to an object dynamically at runtime.
- B) To restrict the creation of an object to a single instance only.
- C) To provide a way to create families of related objects without specifying their concrete classes.
- D) To represent a hierarchical structure where each element is treated the same as others.

Answer: A

Explanation:

- **A (Correct):** The **Decorator Pattern** is designed to add additional responsibilities or behaviours to objects dynamically without altering their structure, making it flexible in terms of functionality.
- **B (Incorrect):** This describes the **Singleton Pattern**, not the Decorator.
- **C (Incorrect):** This explanation fits the **Abstract Factory Pattern**, which deals with creating families of related objects.
- **D (Incorrect):** This describes the **Composite Pattern**, where objects are organized hierarchically to treat individual objects and composites uniformly.

Q#40. What advantages does the Decorator Pattern have over using traditional inheritance? (Select all that apply)

- A) Allows extending behaviour at runtime rather than compile-time.
- B) Provides a way to structure code into base and derived classes.
- C) Avoids creating subclasses for every feature combination.
- D) Ensures thread-safe addition of features.

Answer: A, C

Explanation:

- **A (Correct):** The Decorator Pattern supports runtime extension of behaviours, which is a primary benefit over inheritance.
- **B (Incorrect):** This is true for inheritance but is not specific to decorators.
- **C (Correct):** By using decorators, you can combine features without creating numerous subclasses for each feature combination, unlike inheritance.
- **D (Incorrect):** Decorators don't inherently provide thread-safety for feature addition; thread-safety must be managed separately if needed.

Q#41. In a mobile payment app, the user can select different payment processors like PayPal, Stripe, or a local bank. Each payment processor requires specific steps, yet all follow a core payment flow. Which pattern should you apply?

- A) Template Method Pattern
- B) Strategy Pattern
- C) Proxy Pattern
- D) Observer Pattern

Answer: A) Template Method Pattern

Explanation:

- **(A) Template Method Pattern: Correct.** Template Method defines a core process with processor-specific steps.
- **(B) Strategy Pattern:** Incorrect. Strategy chooses algorithms but doesn't control process flow.
- **(C) Proxy Pattern:** Incorrect. Proxy manages access but isn't for step-specific processes.
- **(D) Observer Pattern:** Incorrect. Observer observes changes, irrelevant to process flow.

Q#42. What is the primary purpose of the Template Method Pattern?

- A) To allow subclasses to override specific parts of an algorithm.
- B) To create an object with a predefined set of properties.
- C) To enforce a single instance of a class.
- D) To provide a way to create complex objects step by step.

Answer: A

Explanation:

- **A (Correct):** The Template Method Pattern defines the structure of an algorithm, allowing subclasses to implement specific steps.
- **B (Incorrect):** This describes a Builder Pattern purpose.
- **C (Incorrect):** This pertains to the Singleton Pattern.
- **D (Incorrect):** This is related to the Builder Pattern, not Template Method.

Q#43. What is the primary purpose of the Mediator Pattern in software design?

- A) To enable communication between classes without needing direct references.
- B) To create a single point of control for class interactions.
- C) To encapsulate how objects interact with each other.
- D) All of the above.

Answer: D

Explanation:

- **Option A:** Correct. The Mediator Pattern allows for communication between classes without them knowing about each other directly.
- **Option B:** Correct. It provides a centralized point for managing interactions, simplifying object relationships.
- **Option C:** Correct. It encapsulates the interaction logic, promoting loose coupling.
- **Option D:** Correct. All listed options capture the essence of the Mediator Pattern.

Q#44. In which scenarios would the Mediator Pattern be beneficial? (Select all that apply)

- A) When multiple classes need to communicate and coordination is required.
- B) When you want to reduce the coupling between classes.
- C) When classes are tightly coupled and dependencies must be maintained.
- D) When you need a single control point to manage the interactions of different objects.

Answer: A, B, D

Explanation:

- **Option A:** Correct. The Mediator Pattern is ideal for scenarios with complex interactions between multiple objects.
- **Option B:** Correct. It reduces the direct dependencies between classes, promoting flexibility.
- **Option C:** Incorrect. The Mediator Pattern aims to reduce tight coupling, not maintain it.
- **Option D:** Correct. It serves as a control point for managing how different objects interact.

Q#45. Which of the following best describes the purpose of the Chain of Responsibility pattern?

- A) It ensures that only one specific handler processes a request.
- B) It allows a sequence of handlers to process a request until one handler succeeds.
- C) It uses multiple handlers to process the same request simultaneously.
- D) It prioritizes the requests based on urgency and processes them accordingly.

Answer: B

Explanation:

- **B is correct** because the Chain of Responsibility pattern is designed to pass a request along a chain of handlers until one of them handles it.
- **A is incorrect** as it implies only one handler is responsible, without passing the request if it fails to handle.
- **C is incorrect** because handlers process requests in sequence, not simultaneously.
- **D is incorrect** as the pattern does not prioritize requests based on urgency by default.

Q#46. Which of the following components is NOT essential in the Chain of Responsibility pattern?

- A) A handler class to process requests
- B) A next handler reference to pass unhandled requests
- C) A client class to initiate the chain
- D) A manager class to prioritize requests

Answer: D

Explanation:

- **D is correct** because the Chain of Responsibility pattern does not require a manager class for prioritization.
- **A, B, and C are incorrect** because they represent core elements: handler classes for processing, a reference to pass requests, and a client to initiate.

Q#47. Which of the following best describes the primary purpose of the Observer Pattern?

- A) To define a one-to-one dependency between objects
- B) To allow an object to be notified when another object changes state
- C) To define the steps involved in a process in a single class
- D) To allow a class to instantiate objects from a family of related classes

Answer: B

Explanation:

- **A (Incorrect):** The Observer Pattern defines a **one-to-many** dependency.
- **B (Correct):** The **Observer Pattern** enables an object (subject) to notify other dependent objects (observers) when its state changes.
- **C (Incorrect):** This describes the **Template Method Pattern**, not Observer.
- **D (Incorrect):** This describes the **Abstract Factory Pattern**, not Observer.

Q#48. An e-commerce platform updates all registered users via email whenever a product's price changes. Which design pattern is most suitable for this scenario?

- A) Singleton Pattern
- B) Observer Pattern
- C) Factory Pattern
- D) Composite Pattern

Answer: B

Explanation:

- **A (Incorrect):** Singleton ensures a class has only one instance, which is unrelated to notification.
- **B (Correct):** The **Observer Pattern** is ideal for notifying multiple users (observers) when the product price (subject) changes.
- **C (Incorrect):** Factory is for object creation, not notification.
- **D (Incorrect):** Composite manages hierarchical object structures, not notifications.

Q#49. An e-commerce site needs to apply different discount approaches based on membership type, like VIP or regular customer. Which pattern best fits to dynamically switch between these approaches?

- A) Strategy Pattern
- B) Adapter Pattern
- C) Flyweight Pattern
- D) Proxy Pattern

Answer: A) Strategy Pattern

Explanation:

- **(A) Strategy Pattern: Correct.** Strategy enables dynamic switching between discount approaches.
- **(B) Adapter Pattern: Incorrect.** Adapter converts interfaces, not intended for choosing approaches.
- **(C) Flyweight Pattern: Incorrect.** Flyweight reduces memory usage but doesn't handle approaches.
- **(D) Proxy Pattern: Incorrect.** Proxy controls access rather than dynamically selecting approaches.

Q#50. What is the primary purpose of the Strategy Pattern?

- A) To provide a way to extend classes in a structured way.
- B) To define a family of algorithms and make them interchangeable at runtime.
- C) To optimize code by reducing the number of classes.
- D) To allow only one instance of a class.

Answer: B

Explanation:

- **Option A:** Incorrect. The Strategy Pattern is not focused on extending classes or inheritance structures but on providing interchangeable behaviours, each represented as a separate class implementing a specific strategy interface.
- **Option B: Correct.** The Strategy Pattern defines a family of algorithms, each encapsulated in its own class, and makes it possible to switch between them at runtime without changing the client code. This flexibility allows different implementations to be chosen based on dynamic conditions.
- **Option C:** Incorrect. The Strategy Pattern may lead to additional classes, as each strategy typically requires its own concrete implementation. Its goal is not to reduce the number of classes but to allow various algorithms to be selected and used interchangeably.
- **Option D:** Incorrect. Limiting class instances is the purpose of the Singleton Pattern, not the Strategy Pattern. The Strategy Pattern focuses on dynamically changing behaviour rather than enforcing a single instance.

Q#51. What is the primary purpose of the Command Pattern?

- A) To encapsulate commands as objects, allowing them to be parameterized
- B) To enforce direct communication between objects for high performance
- C) To enable classes to inherit behaviour from multiple parents
- D) To facilitate automatic dependency injection across objects

Answer: A

Explanation:

- **A is correct** because the Command Pattern encapsulates a request as an object, allowing it to be parameterized and used flexibly.
- **B is incorrect** as the pattern does not focus on direct communication between objects.
- **C is incorrect** since inheritance is unrelated to the Command Pattern.
- **D is incorrect** because dependency injection is not a primary concern of this pattern.

Q#52. In which scenarios would the Command Pattern be most beneficial?
(Select all that apply)

- A) Implementing an undo/redo system for a text editor
- B) Building a direct-messaging feature for a social media app
- C) Developing a menu system with various operations in a graphical interface
- D) Creating a singleton service for database connections

Answer: A and C

Explanation:

- **A and C are correct** because the Command Pattern is effective for implementing undo/redo functionalities and encapsulating operations in menu systems.
- **B is incorrect** since a direct-messaging feature does not require command encapsulation.
- **D is incorrect** as it pertains to singleton usage rather than command encapsulation.

Q#53. What is the primary purpose of the State Pattern in object-oriented design?

- A) To encapsulate multiple states within a single object
- B) To separate behaviour based on an object's state into different classes
- C) To store the history of state changes in an application
- D) To enable communication between objects in different states

Answer: B

Explanation:

- **B is correct** because the State Pattern allows behaviour to vary based on an object's state by delegating to different classes for each state.
- **A is incorrect** as it doesn't encapsulate multiple states within a single object; rather, it dynamically changes behaviour.
- **C is incorrect** since tracking state history is not the main purpose of the State Pattern.
- **D is incorrect** as inter-object communication isn't typically part of this pattern's goal.

Q#54. A media player application has different states: Playing, Paused, and Stopped. Which design pattern would best manage transitions between these states?

- A) Observer Pattern
- B) State Pattern
- C) Command Pattern
- D) Factory Pattern

Answer: B

Explanation:

- **B is correct** because the State Pattern is designed to handle an object's behaviour based on its current state, such as "Playing," "Paused," or "Stopped."

- **A, C, and D** are **incorrect** as they don't focus on managing state transitions and state-based behaviour.

Q#55. What is the primary purpose of the Visitor Pattern?

- A) To simplify state transitions between objects
- B) To define a new operation on an object without changing its class
- C) To create a single class that handles multiple requests
- D) To enforce strict access control within an object structure

Answer: B

Explanation:

- **B** is **correct** because the Visitor Pattern allows adding new operations to classes without modifying them.
- **A, C, and D** are **incorrect** as they do not describe the purpose of the Visitor Pattern.

Q#56. A company has an employee hierarchy with roles such as Manager, Engineer, and Intern. It needs to perform various reports on these roles, such as performance analysis and salary calculation. Why might the Visitor Pattern be beneficial here? (Select all that apply)

- A) It allows adding new reporting functionalities without modifying existing role classes
- B) It centralizes role-specific data and makes it easier to manage roles
- C) It encourages the separation of roles and operations on them
- D) It enforces role-based security and access controls

Answer: A and C

Explanation:

- **A and C** are **correct** because the Visitor Pattern allows adding new operations (e.g., performance analysis) without altering the existing role classes, adhering to open/closed principle.
- **B and D** are **incorrect** as they do not represent the pattern's focus on operations over structure.

Q#57. What is the primary purpose of the Iterator Pattern in an object-oriented design?

- A) To support cloning objects within a collection
- B) To enable the creation of a single instance of a class

- C) To allow for the lazy instantiation of objects
- D) To provide a way to access elements of a collection sequentially without exposing its underlying structure

Answer: D

Explanation:

- **D is correct** as the Iterator Pattern enables sequential access to collection elements without exposing the internal structure.
- **A is incorrect** as cloning is managed by the Prototype Pattern.
- **B is incorrect** because the Singleton Pattern is used for creating single instances.
- **C is incorrect** because lazy instantiation is related to the Lazy Initialization Pattern.

Q#58. What is the primary purpose of the Interpreter pattern in design patterns?

- A) To interpret user input in a GUI application
- B) To define a representation for a language's grammar and provide a way to evaluate sentences in that language
- C) To create object hierarchies based on different levels of interpretation
- D) To manage complex conditional expressions in a program

Answer: B

Explanation:

- **B is correct** because the Interpreter pattern is designed to represent the grammar of a language and to interpret sentences in that language.
- **A is incorrect** as the Interpreter pattern is not specifically for GUI inputs.
- **C is incorrect** because the pattern does not focus on object hierarchies but rather on grammar.
- **D is incorrect** as the pattern is not intended for managing conditional expressions but for creating interpreters.

Q#59. Which of the following best describes the Memento pattern in design?

- A) Capturing and externalizing an object's internal state so it can be restored later without violating encapsulation
- B) Defining a family of algorithms and encapsulating each one in a separate object
- C) Representing the dependencies between objects to establish dynamic connections

D) Converting the interface of a class into another interface expected by the client

Answer: A

Explanation:

- **A is correct** as the Memento pattern is used to capture and restore an object's state without exposing its implementation.
- **B is incorrect** as this describes the Strategy pattern.
- **C is incorrect** as this describes the Observer pattern.
- **D is incorrect** as this describes the Adapter pattern.

Q#60. In the Memento pattern, which of the following are key roles? (Select two)

- A) Originator
- B) Handler
- C) Caretaker
- D) Adapter

Answer: A, C

Explanation:

- **A is correct** because the Originator is responsible for creating and restoring states.
- **C is correct** because the Caretaker manages the Memento but does not modify it.
- **B is incorrect** as there is no Handler role in the Memento pattern.
- **D is incorrect** as Adapter is a different pattern.



Want to Master Java Design Patterns?

You have just explored a basic set of Java Design Patterns interview questions, MCQs with explanations. But this is just the beginning!

Join Udemy Course for Practicing Interview Questions



-  **Practice Mode** to learn interactively with instant feedback
-  **Exam Mode** to simulate real-time interviews or assessments
-  **600+ Questions with Answers & 1800+ concept explanations**
-  **Covers all 23 GoF (Gang of Four) design patterns with real-world Java examples.**
-  **Concept-based, Scenario-based, Hands-On Coding-based Questions**
-  **Interview-Focused Content for all interviews**
-  **Includes Dual (Practice & Exam) Mode for complete preparation.**
-  **Lifetime Access with Certification**
-  **Learn Anytime, Anywhere**
-  **Community Q&A + Instructor Support**
-  **Value for Money**

Practice Mode vs Exam Mode (Included in the Course!)

Feature	Practice Mode	Exam Mode
Purpose	Learn by solving questions at your pace	Test your knowledge under real-time pressure
Timing	No time limit	Time-bound (simulates real exams)
Feedback	Immediate answer explanations	Feedback after completing the set
Flexibility	Pause, retry, and review	One-shot attempt like in real exams
Best For	Beginners and revision sessions	Interview preparation and assessment

 **Enroll Now:**

[Java Design Patterns Interview Practice Test & Interview Questions.](#)

 **Get the Complete Kindle eBook**



-  500+ Questions with Answers & 1500+ concept explanations
-  Questions Containing Code Snippets
-  Designed for Beginners to Experts
-  Read Anytime, Anywhere
-  Structured Learning Path
-  Portable and Offline Access
-  One-Time Purchase – Lifetime Learning
-  Works on All Devices
-  Verified on Amazon

 **Get from Amazon:**

For amazon.in users:

[Master Java Design Patterns with 500+ Solved MCQs & 1500+ Concepts Explanations](#)

For amazon.com users:

[Master Java Design Patterns with 500+ Solved MCQs & 1500+ Concepts Explanations](#)

Any other Amazon Users, kindly search product by ASIN 'BOF4Y467PL' or by text 'Java Design Patterns for Real-World Expertise'

 Want to learn complete concepts of all Design Patterns with Examples?

Kindly visit: [Java Design Patterns Complete Tutorial](#)

 Practice Set for Other Java Related Technologies: [Java Question Hub](#)

✉ For support or queries, reach out at: javatechonline@gmail.com 🧠
Keep learning. Keep building. Master patterns like a pro.

javatechonline.com